

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence    COURSE CODE: CSA1708**

---

### COURSE OUTCOME COVERED

**CO4:** *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

*Assessment Tool Weightage: 40%*

---

**QUESTIONS: 2**

**Duration : 45 Minutes**  
**Total Marks:20**

Annexure A

### INTEGRATED EXPERIMENTS

---

#### ***Code of Conduct:***

*I \_\_\_\_M Kavya 192472370\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: kavya***

## INTEGRATED EXPERIMENTS

---

### **Experiment 1:** Decision Tree Classification

**Objective:** Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

### **Experiment 2:** Reinforcement Learning – Q-Learning

**Objective:** Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters ( $\alpha$ ,  $\gamma$ ,  $\epsilon$ ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

## Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python (scikit-learn).

Tools used: Python 3, pandas, scikit-learn, matplotlib.

### (a) Data Loading and Pre-processing

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
iris = load_iris(as_frame=True)
df = iris.frame.copy()
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))

print(df.head())      # first 5 rows
print(df.dtypes)      # data types
print(df.isnull().sum()) # check missing values

# No missing values / categorical encoding needed - Iris is fully numeric
X = df[iris.feature_names]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

**Output — First 5 rows:**

| sepal length | sepal width | petal length | petal width | target | species |
|--------------|-------------|--------------|-------------|--------|---------|
| 5.1          | 3.5         | 1.4          | 0.2         | 0      | setosa  |
| 4.9          | 3.0         | 1.4          | 0.2         | 0      | setosa  |
| 4.7          | 3.2         | 1.3          | 0.2         | 0      | setosa  |
| 4.6          | 3.1         | 1.5          | 0.2         | 0      | setosa  |
| 5.0          | 3.6         | 1.4          | 0.2         | 0      | setosa  |

**Output — Data types:**

| Column            | Data Type       |
|-------------------|-----------------|
| sepal length (cm) | float64         |
| sepal width (cm)  | float64         |
| petal length (cm) | float64         |
| petal width (cm)  | float64         |
| target            | int64           |
| species           | object (string) |

**Output — Train/test split and missing-value check:**

Training samples: 105, Testing samples: 45  
 Missing values per column: 0 (no missing values in any column)

The Iris dataset has no missing values and all features are already numeric, so no imputation or categorical encoding was required. The target column is a label-encoded integer (0 = setosa, 1 = versicolor, 2 = virginica). The data was split into 105 training samples and 45 testing samples (70/30 split) using stratified sampling to preserve the class balance across all three species.

**(b) Building the Decision Tree (Gini Index)**

```
from sklearn.tree import DecisionTreeClassifier, export_text

clf = DecisionTreeClassifier(criterion="gini", random_state=42, max_depth=4)
clf.fit(X_train, y_train)

print(export_text(clf, feature_names=list(X.columns)))
print("Feature importances:", dict(zip(X.columns, clf.feature_importances_)))
```

**Output — Tree structure:**

```
|--- petal length (cm) <= 2.45
| |--- class: 0 (setosa)
|--- petal length (cm) > 2.45
| |--- petal width (cm) <= 1.55
| | |--- petal length (cm) <= 4.95
| | | |--- class: 1 (versicolor)
| | |--- petal length (cm) > 4.95
| | | |--- class: 2 (virginica)
| |--- petal width (cm) > 1.55
| | |--- petal width (cm) <= 1.70
| | | |--- sepal width (cm) <= 2.85
```

```

| | | | |--- class: 1 (versicolor)
| | | |--- sepal width (cm) > 2.85
| | | |--- class: 2 (virginica)
| | |--- petal width (cm) > 1.70
| | |--- petal length (cm) <= 4.85
| | | |--- class: 1 (versicolor)
| | | |--- petal length (cm) > 4.85
| | | |--- class: 2 (virginica)

```

Root node: petal length (cm) <= 2.45

Feature importances: petal length=0.549, petal width=0.436, sepal width=0.014, sepal length=0.000

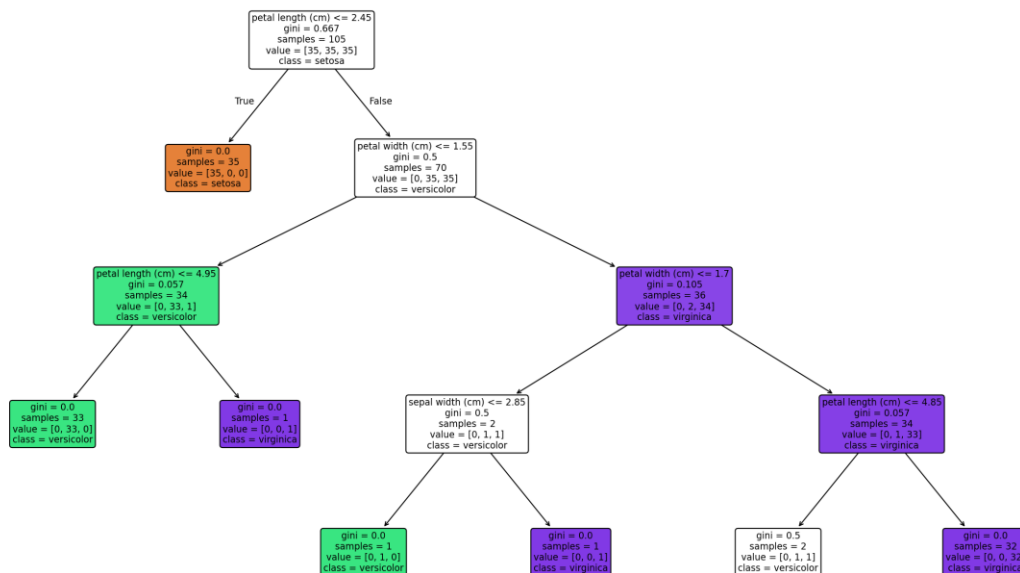


Figure 1: Graphical structure of the trained Decision Tree (Gini index, max\_depth=4).

### Root Node Justification

The root node splits on petal length (cm) <= 2.45. This attribute was selected automatically by the CART algorithm because it produces the largest reduction in Gini impurity of any single split: the pre-split Gini impurity is 0.667 (three equally sized classes of 35 samples each), and splitting on petal length <= 2.45 perfectly isolates all 35 setosa samples into a pure leaf (Gini = 0.0), while the remaining 70 samples (versicolor and virginica) fall into the other branch. The computed feature importances confirm this: petal length (0.549) and petal width (0.436) dominate, while sepal width (0.014) and sepal length (0.000) contribute almost nothing, consistent with the well-known biological fact that petal measurements separate Iris species far better than sepal measurements.

### (c) Model Evaluation

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

```
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

**Output — Accuracy: 0.8889 (88.89%)**

**Output — Confusion Matrix:**

| Actual \ Predicted | setosa | versicolor | virginica |
|--------------------|--------|------------|-----------|
| setosa             | 15     | 0          | 0         |
| versicolor         | 0      | 12         | 3         |
| virginica          | 0      | 2          | 13        |

**Output — Classification Report:**

| Class        | Precision | Recall | F1-Score | Support |
|--------------|-----------|--------|----------|---------|
| setosa       | 1.00      | 1.00   | 1.00     | 15      |
| versicolor   | 0.86      | 0.80   | 0.83     | 15      |
| virginica    | 0.81      | 0.87   | 0.84     | 15      |
| Accuracy     |           |        | 0.89     | 45      |
| Macro avg    | 0.89      | 0.89   | 0.89     | 45      |
| Weighted avg | 0.89      | 0.89   | 0.89     | 45      |

**Misclassified instances (5 of 45 test samples):**

| Idx | Sepal L | Sepal W | Petal L | Petal W | Actual     | Predicted  |
|-----|---------|---------|---------|---------|------------|------------|
| 56  | 6.3     | 3.3     | 4.7     | 1.6     | versicolor | virginica  |
| 138 | 6.0     | 3.0     | 4.8     | 1.8     | virginica  | versicolor |
| 85  | 6.0     | 3.4     | 4.5     | 1.6     | versicolor | virginica  |
| 77  | 6.7     | 3.0     | 5.0     | 1.7     | versicolor | virginica  |
| 106 | 4.9     | 2.5     | 4.5     | 1.7     | virginica  | versicolor |

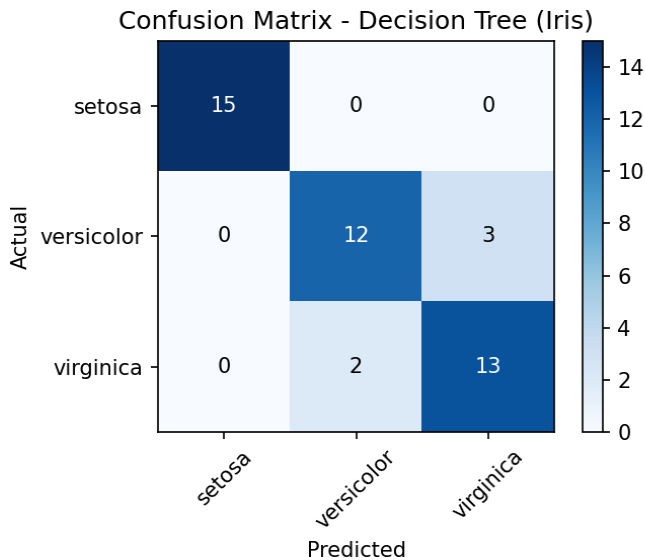


Figure 2: Confusion matrix heat-map for the Decision Tree classifier on the test set.

### Performance Comment

The model achieved 88.89% accuracy on the held-out test set. All 15 setosa samples were classified perfectly (precision = recall = 1.00), confirming that setosa is linearly separable from the other two species on petal measurements alone. The confusion is concentrated entirely between versicolor and virginica: 3 versicolor samples were predicted as virginica and 2 virginica samples were predicted as versicolor, giving versicolor a recall of 0.80 and virginica a recall of 0.87. This is expected because these two species have overlapping petal-width and petal-length ranges around the 1.6–1.8 cm boundary, as seen in the two misclassified rows with petal width 1.6–1.8 cm sitting close to the tree's decision thresholds (1.55 and 1.70). Increasing `max_depth` or using an ensemble method such as Random Forest would likely resolve some of this boundary confusion.

### Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4x4 Grid World environment and observe the agent's learned policy.

Tools used: Python 3, NumPy, matplotlib.

#### (a) Environment Definition

```
import numpy as np

GRID_SIZE = 4
ACTIONS = ["Up", "Down", "Left", "Right"]
GOAL_STATE = 15 # bottom-right corner
OBSTACLE_STATES = [5, 7, 11] # obstacle cells
START_STATE = 0

ALPHA, GAMMA, EPSILON = 0.1, 0.9, 0.2 # learning rate, discount, exploration
EPISODES, MAX_STEPS = 100, 50

Q = np.zeros((16, 4)) # Q-table initialised with zeros
```

```

def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == "Up": r = max(0, r - 1)
    if action == "Down": r = min(GRID_SIZE - 1, r + 1)
    if action == "Left": c = max(0, c - 1)
    if action == "Right": c = min(GRID_SIZE - 1, c + 1)
    next_state = r * GRID_SIZE + c
    if next_state == GOAL_STATE: reward, done = 10, True
    elif next_state in OBSTACLE_STATES: reward, done = -5, False
    else: reward, done = -1, False
    return next_state, reward, done

```

### ***Output — Environment setup:***

```

Grid: 4x4, States: 16, Actions: ['Up','Down','Left','Right']
Start: 0, Goal: 15, Obstacles: [5, 7, 11]
Reward structure: Goal=+10, Step=-1, Obstacle=-5
Hyperparameters: alpha=0.1, gamma=0.9, epsilon=0.2
Initial Q-table: 16x4 matrix of zeros

```

The grid world has 16 states (indexed 0–15, row-major) and 4 actions (Up, Down, Left, Right). The agent starts at state 0 (top-left) and must reach the goal at state 15 (bottom-right), earning +10 reward. Each ordinary move costs -1 (to encourage shorter paths), and stepping onto one of the three obstacle cells (states 5, 7, 11) incurs a -5 penalty without ending the episode. Boundary moves that would leave the grid simply keep the agent in place. The Q-table is initialised to all zeros (a 16x4 matrix) and updated using the hyperparameters: learning rate  $\alpha = 0.1$ , discount factor  $\gamma = 0.9$ , and exploration rate  $\epsilon = 0.2$  (epsilon-greedy action selection).

### **(b) Running Q-Learning (100 Episodes)**

```

for episode in range(1, EPISODES + 1):
    state = START_STATE
    for _ in range(MAX_STEPS):
        if np.random.rand() < EPSILON:
            a = np.random.randint(4)          # explore
        else:
            a = np.argmax(Q[state])           # exploit
        next_state, reward, done = step(state, ACTIONS[a])
        Q[state, a] += ALPHA * (reward + GAMMA * np.max(Q[next_state]) - Q[state, a])
        state = next_state
    if done:
        break

```

The Q-values for two representative states — State 0 (the start state) and State 10 (an interior state adjacent to the goal path) — were recorded at episodes 1, 50, and 100 to observe how the table evolves:



| Episode | State     | Up     | Down   | Left   | Right  |
|---------|-----------|--------|--------|--------|--------|
| 1       | 0 (Start) | -0.199 | -0.199 | -0.271 | -0.100 |
| 1       | 10        | -0.100 | 0.000  | 0.000  | 0.000  |
| 50      | 0 (Start) | -2.279 | -2.047 | -2.249 | -2.163 |
| 50      | 10        | -0.303 | 7.462  | 0.096  | -0.986 |
| 100     | 0 (Start) | -2.071 | 1.168  | -2.308 | -2.240 |
| 100     | 10        | 0.506  | 7.992  | 1.825  | -0.821 |

At Episode 1, nearly all Q-values are close to zero or slightly negative, since the agent has barely explored the environment and most state-action pairs have only been updated once or not at all. By Episode 50, State 10's Down action has risen sharply to 7.462 — this is the action that leads directly toward the goal, and its Q-value is approaching the discounted goal reward. State 0's values are still deeply negative at this stage because reaching the goal from the start requires several steps, so the -1 step penalty dominates early in training before the +10 goal reward has propagated backward through the table. By Episode 100, State 10 Down has increased further to 7.992 (near convergence), and State 0's Down action has become the clear best choice (1.168, now positive), showing that the value of the +10 goal reward has propagated back through the chain of states toward the start as training progressed.

### (c) Final Policy and Convergence

```

policy = {}
for s in range(16):
    if s == GOAL_STATE or s in OBSTACLE_STATES:
        continue
    policy[s] = ACTIONS[np.argmax(Q[s])]
print(policy) # final learned optimal policy per state

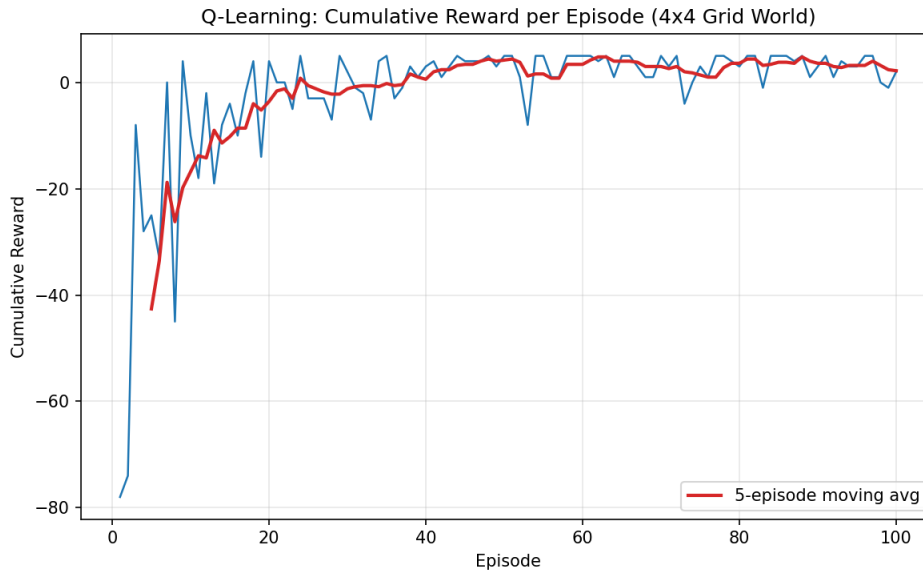
```

### *Output — Final learned policy and reward summary:*

Final policy (row, col -> best action):  
Row0: Down Right Down Left  
Row1: Down X Down X  
Row2: Right Right Down X  
Row3: Right Right Right G (Goal)

Path from Start(0): Down->4, Down->8, Right->9, Right->10, Down->14, Right->15(Goal)  
- avoids all three obstacle cells (5, 7, 11)

Average reward, first 10 episodes: -29.70  
Average reward, last 10 episodes: 2.70



*Figure 3: Cumulative reward per episode, with a 5-episode moving average, showing convergence over 100 episodes.*

### **Convergence Justification**

The cumulative reward per episode starts strongly negative (around -78 in episode 1, averaging -29.70 over the first 10 episodes), reflecting an agent that wanders inefficiently, repeatedly hitting walls, obstacles, and taking long paths before it randomly discovers the goal. As episodes progress, the reward trend rises steadily and stabilises around small positive values by the final episodes (averaging +2.70 over the last 10 episodes), which corresponds to reaching the goal in close to the minimum number of steps (6 steps: +10 for the goal minus 6 x -1 for the steps = +4, reduced slightly by the residual epsilon = 0.2 exploration that still occasionally causes detours). The extracted final policy traces a clean path from the start (state 0) to the goal (state 15) — Down, Down, Right, Right, Down, Right — that successfully avoids all three obstacle cells, confirming that the Q-learning agent converged to a near-optimal policy within 100 episodes.

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                                                    | <b>Level 3 – Proficient</b>                                 | <b>Level 2 – Developing</b>                         | <b>Level 1 – Beginning</b>                       |
|---------------------------|------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25               | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30               | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20               | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15               | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10               | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |